

Reader's Guide to HATPro Development Docs (ZIP Set)

This guide consolidates the **developer guide documents in DevelopmentGuideManuals.zip** into an opinionated map of:

- the **major topics covered by the set** (with minimal duplication), and
- **where to go for details** (which document is the “best” reference for a topic, considering overlap and last-modified dates).

Notes on overlap: several documents repeat the same core ideas at different depths (quick guide vs manual, or MD vs DOCX variants). Where duplicates exist, this guide identifies a **preferred “primary” doc** and treats the rest as supporting or legacy copies.

1) Document inventory (with last-modified date)

Sorted newest → oldest (ZIP file timestamps):

1. **HATPro Development Phases Manual.md** — 2025-11-28
 2. **README_WIP.md** — 2025-11-27
 3. **The Work-in-Progress (WIP) Approach.md** — 2025-11-27
 4. **HATpro Schema Pipeline Readme.md** — 2025-11-21
 5. **Draft Merge Plan - Puml models to JSON Schema.docx** — 2025-11-21
 6. **Hatpro Dev Phases Manual.md** — 2025-11-19
 7. **The Build a Profile Assembly Line.docx** — 2025-11-19
 8. **Hatpro Dev Phases Manual.docx** — 2025-11-19
 9. **HATPro - Validation_Testing_Manual.docx** — 2025-11-10
 10. **HATPro Developers Guide.docx** — 2025-11-10
 11. **HATPro source code as Training Material.docx** — 2025-11-10
 12. **Using NPM for HATPro Development.docx** — 2025-11-05
-

2) Major topics covered by the set (the “integrated” overview)

Across the full set, the content clusters into these major topics:

A. Repository layout and artifact taxonomy

What lives where in the repo (PUML sources, generated schemas, enums, samples, templates/examples), and why that separation matters.

B. Toolchain / pipeline

The end-to-end lifecycle from **PUML** → **JSON Schema / Enum JSON** → **compilation** → **validation** → **samples/templates/tests** → **CI**.

C. Schema authoring conventions

How to write PUML for deterministic codegen, including:

- how **\$id** and **\$ref** are formed and resolved
- SCHEMAHINTS / ENUMHINTS directives
- enum authoring conventions
- anti-patterns and troubleshooting

D. Validation and testing

How AJV is used, what's validated (schemas vs enums vs examples), test categories, and CI recommendations.

E. Migration/merge plan (bringing older models forward)

A practical plan to take existing PUML (from the last year) and move it into the current layout and conventions **without breaking generators**.

F. Work-in-progress strategy for large changes

A workstation WIP / snapshot approach to protect against risky refactors and allow multi-day changes without polluting Git history.

G. “Build a profile” assembly line concept

The conceptual blueprint for generating not just schemas, but also templates and “profile-building” workflows (segment builders → profile assembler → validator).

H. NPM basics for the project

A beginner-friendly explanation of Node/npm concepts and what commands to run (especially Windows-oriented).

I. Using repo content as training material

How to tag “good examples” in-tree and index them for onboarding/training.

3) Topic → where to go (primary references)

This is the most important section: **if you want to learn or implement X, start with Y**.

A) Repo layout + artifacts + pipeline (start here)

Primary: *HATPro Development Phases Manual.md* (newest, unified “manual + cheatsheet”)

Secondary: *HATpro Schema Pipeline Readme.md* (shorter; folder taxonomy + suggested npm scripts)

B) Authoring conventions: \$id / \$ref, SCHEMAHINTS, ENUMHINTS

Primary: *HATPro Developers Guide.docx* (deepest reference; worked examples; anti-patterns; troubleshooting)

C) Validation & testing (AJV, examples, CI)

Primary: *HATPro - Validation_Testing_Manual.docx*

Secondary: *HATPro Development Phases Manual.md* (pipeline view; where validation fits)

D) Merging older models into the pipeline

Primary: *Draft Merge Plan - Puml models to JSON Schema.docx*

(Goals, phases, guardrails, risk/mitigation, daily loop, handoff checklist.)

E) Large change management (WIP strategy)

Primary (quick): *README_WIP.md*

Primary (full explanation): *The Work-in-Progress (WIP) Approach.md*

F) Building templates / “profile builder” concept

Primary: *The Build a Profile Assembly Line.docx*

(Clarifies schema vs samples vs templates; proposes segment templates and an assembly pipeline.)

G) NPM fundamentals (especially for new contributors)

Primary: *Using NPM for HATPro Development.docx*

H) Training / discoverability via in-repo examples

Primary: *HATPro source code as Training Material.docx*

4) Recommended reading paths (depending on who you are)

Path 1 — New contributor who will run tools (not change generators)

- 1) *HATpro Schema Pipeline Readme.md* (orientation, folders, scripts)
- 2) *Using NPM for HATPro Development.docx* (get running quickly)

- 3) *HATPro Development Phases Manual.md* (understand phases + checks)
- 4) *HATPro - Validation_Testing_Manual.docx* (how we prove correctness)

Path 2 — PUML author who needs deterministic codegen

- 1) *HATPro Developers Guide.docx* (SCHEMAHINTS, ENUMHINTS, *id/ref*)
- 2) *HATPro Development Phases Manual.md* (pipeline expectations)
- 3) *HATPro - Validation_Testing_Manual.docx* (what will break + why)

Path 3 — Maintainer doing a large refactor or big model merge

- 1) *Draft Merge Plan - Puml models to JSON Schema.docx* (the plan)
- 2) *The Work-in-Progress (WIP) Approach.md* + *README_WIP.md* (safety net)
- 3) *HATPro Development Phases Manual.md* (verification loop)

Path 4 — “Profile builder” / template generation / UX-oriented tooling

- 1) *The Build a Profile Assembly Line.docx*
 - 2) *HATpro Schema Pipeline Readme.md*
 - 3) *HATPro Developers Guide.docx* (only as needed for hint semantics)
-

5) Per-document quick summaries (what it contains; how it differs)

5.1 HATPro Development Phases Manual.md (2025-11-28)

- Unified description of the **end-to-end phases**: lint → generate → validate → verify.
- Clarifies how phases apply differently to **PUML classes vs PUML enums vs generated artifacts vs samples/examples**.
- Best “single place” to understand what the pipeline is supposed to do.

5.2 README_WIP.md (2025-11-27)

- Fast, operational cheat sheet for the **Workstation WIP snapshot strategy**.
- Lists snapshot commands and folder naming conventions.

5.3 The Work-in-Progress (WIP) Approach.md (2025-11-27)

- Full rationale and explanation behind WIP snapshots.
- When to use them, how they complement Git branches, and why HATPro needs this pattern.

5.4 HATpro Schema Pipeline Readme.md (2025-11-21)

- Repo **artifact taxonomy** and folder structure (schemas, enums, samples, templates, tests).
- Suggests a practical package .json scripts block for common tasks.

5.5 Draft Merge Plan - Puml models to JSON Schema.docx (2025-11-21)

- Stepwise merge plan to bring existing PUMML into the target layout.
- Includes success criteria, guardrails for \$id/\$ref, lint/validation gates, and risk mitigation.

5.6 Hatpro Dev Phases Manual.md (2025-11-19)

- Appears to be an earlier/duplicate copy of the development phases manual.
- Treat as **secondary** unless it contains a specific section not present in the 2025-11-28 manual.

5.7 The Build a Profile Assembly Line.docx (2025-11-19)

- Distinguishes **schema vs samples vs templates**, and proposes two template types:
 - structural templates derived from schema
 - catalog-expanded templates (schema + catalogs)
- Frames profile creation as an “assembly line”: parts → segments → segment builders → master profile builder.

5.8 Hatpro Dev Phases Manual.docx (2025-11-19)

- DOCX variant of the development phases manual; likely redundant with the MD versions.

5.9 HATPro - Validation_Testing_Manual.docx (2025-11-10)

- AJV usage and what “validation” means in this repo.
- Separates generated artifact verification vs developer-authored example instance tests.
- CI recommendations + troubleshooting.

5.10 HATPro Developers Guide.docx (2025-11-10)

- The deepest “how to author for codegen” reference.
- \$id/\$ref formation and referencing conventions.
- SCHEMAHINTS and ENUMHINTS details + mapping rules + anti-patterns.
- Includes worked examples for enum generation and referencing.

5.11 HATPro source code as Training Material.docx (2025-11-10)

- Proposes “living examples in-tree” with lightweight metadata tags.
- Suggests an indexing approach so contributors can find good patterns quickly.

5.12 Using NPM for HATPro Development.docx (2025-11-05)

- Node/npm fundamentals: dependencies vs devDependencies.
- Practical guidance for Windows setup and daily workflow.

- Best for onboarding contributors unfamiliar with npm.
-

6) Editorial recommendations (to reduce future overlap)

If you want this doc set to age well with less duplication:

1. Declare **one canonical manual** for pipeline phases and archive older copies.
 - Likely: keep **HATPro Development Phases Manual.md** as the source of truth.
 2. Keep **HATPro Developers Guide.docx** as the canonical deep reference for SCHEMAHINTS / ENUMHINTS / *id* / ref.
 3. Keep WIP docs split as they are (quick vs full).
 4. Consider adding a top-level docs/README.md that links:
 - “Start here” → pipeline readme + phases manual
 - “Authoring conventions” → developers guide
 - “Validation & CI” → validation manual
 - “Big changes” → merge plan + WIP strategy
-

7) What this guide intentionally does NOT do

- It does **not** rewrite all content into one monolithic document.
- It does **not** attempt to reconcile every wording difference; it picks a **primary reference per topic** and treats the rest as supporting.

That matches your goal: consolidate at the **topic level**, and point to specific docs for detailed procedure and “source of truth.”